

COMP2000

Software Engineering 2

20 CREDIT MODULE

ASSESSMENT: 100% Coursework **W1: 30% Design Exercises**
W2: 70% Implementation

MODULE LEADER: Dr Vivek Singh

MODULE AIMS

- To learn about topics that instil best practice into the students' software development activity.
- To explore a range of commonly used programming paradigms.
- To understand the benefit of using standardised design patterns.

ASSESSED LEARNING OUTCOMES (ALO):

1. Apply HCI and parallelism to construct more efficient and usable software.
2. Illustrate the relative merits of various programming paradigms
3. Use appropriate programming paradigms to implement software that solves a given problem.
4. Compare standard design patterns and explain why they are an important aspect of software engineering

Overview

This document contains all the necessary information pertaining to the assessment of COMP2000 (Software Engineering 2). The module is assessed via 100% coursework, across two elements:

1. 30% Design Exercises
2. 70% Implementation

The sections that follow will detail the assessment tasks that are to be undertaken. The submission and expected feedback dates are presented in Table 1. All assessments are to be submitted electronically via the respective DLE module pages before the stated deadlines.

	Submission Deadline	Feedback
Design Report (30%)	3 November 2025 (15:00)	2 December 2025 (15:00)
Implementation Report (70%)	9 January 2026 (15:00)	6 February 2026 (15:00)

Table 1: Assessment Deadlines

All assessments will be introduced in class to provide further clarity over what is expected and how you can access support and formative feedback prior to submission. **Whilst the assessment information is provided at the start of the module, it is not necessarily expected you will start this immediately** – as you will often not have sufficient understanding of the topic. The module leader will provide guidance in this respect.

Moderation

This assignment brief has been moderated in line with the university policy.

Moderator Name	Moderation Date
Shaymaa Al-Juboori	08/09/2025

Useful Assignment Information

Please see below some useful information regarding submissions for your modules.

Good Coding Practices

Like all things, no code is perfect. Just because your code compiles and runs does not mean it is perfect, there is always room for improvement. No code will achieve 100% of the available marks. Where code submission is required by a module, it will be assessed against the following criteria:

1) Functionality

- a. Does your code meet all the specified requirements of the assignment?
- b. Does your code behave correctly across typical and edge-case scenarios?
- c. Does your code have appropriate error handling that does not lead to it crashing or undefined behaviour?

2) Efficiency

- a. Is your code optimised in terms of performance and resource use?
- b. Does your code handle functions, variables, data calls efficiently?
- c. Is there any unnecessary repetition, complexity or processing reducing efficiency within your code?

3) Readability

- a. Is your code logically structured and easy to read?
- b. Have you maintained standard formatting conventions like indentations, spacing and naming, within your code?
- c. Are variables, functions, and classes clearly named and purposeful?

4) Documentation

- a. Are there helpful comments explaining non-obvious parts of your code?
- b. Do your comments document your process, development or rationale clearly?
- c. Could someone unfamiliar with the code understand the approach you have taken?

Use of Generative AI for Creating Code

Each assignment element for each module will have a clear indication of the permitted level of use of AI (solo, assisted or partnered) including the generation of code. Please ensure that you **read and understand the permitted use**. If you are unsure of a particular use, please reach out to the Module Leader and ask.

We strongly encourage you to read and explore beyond the core content delivered within the modules. However, this must be done correctly following academic process. Wherever code is drawn from an outside source (e.g. you have not written it yourself), regardless of whether this is AI generated or from an online repository (GitHub, Stack Overflow etc) **you must reference the original source**. This can be done in your documentation as comments or part of your write ups. Students found to be utilising code without referencing could face an academic offence. **If in doubt speak to your Module Leader.**

Versioning

A critical part of a development (software or academic) is versioning. Keeping a number of iterations over the course of development ensures you always have backups to fall back on. It also provides clear demonstration of the development process you implemented.

Using online/cloud-based storage solutions and repositories provide additional peace of mind, alongside being industry practice. The university provide OneDrive to all students which offers inbuilt version history for Microsoft products. GitHub is the university recommended repository for versioning code, which includes great integration with a number of IDEs.

Submitting Code

It is vital that you confirm that all code that you submit is in the **correct format** and **compiles correctly** or **runs without error**. If you clean up your code before submission, please ensure that all dependencies (libraries, functions and variables) are included so that the marker can compile your code.

We can only mark what has been submitted. If the code does not run correctly, it is not our responsibility to spend time error handling to award you marks.

It is also important to **show your working**. Tidying up your code is a critical part of coding practices, but if you remove the workings that provides you with the required output, we cannot see how you got there.

In a worst-case scenario, if you remove a key part of your working, and on compiling your code it gives an error or different result, we have no way of confirming what went wrong, or indeed if you fabricated those outputs.

Please note, just because a piece of code has given the wrong output, it does not mean it should be deleted. Seeing these attempts with comments documenting your attempt allows us to understand where you may have made an error and provide feedback that addresses this. In some cases, you could also be awarded marks for the attempt.

Coursework Scenario: Restaurant Management Application

A restaurant chain wants to develop a mobile application to manage its restaurant operations more efficiently. The app will have two user types: **Staff** and **Guest**. The staff side will allow restaurant employees to manage menu items and reservations, while the guest side will enable customers to browse the menu and make table reservations.

Both staff and guests must log in to access the application's main features. The application will interact with a **RESTful API** that provides secure access to the database of the app users via standard HTTP methods. Menu and reservations should be managed locally on the device using local database (preferably **SQLite**).

The app should show notifications to inform users of relevant updates. For example:

- Guests get notified when their table booking is modified or cancelled.
- Staff receive notifications for new reservations and change of reservations.
- Users should also have the ability to turn notifications on or off and manage their preferences in their account settings.

Staff Side

This side of the application is for restaurant employees who need to manage menu items and handle customer reservations.

Functional Requirements – Staff

- As a staff member, I wish to add new menu items with details (name, price, and image).
- As a staff member, I wish to edit menu items when details change.
- As a staff member, I wish to delete menu items that are no longer available.
- As a staff member, I wish to view and cancel customer reservations.
- As a staff member, I wish to receive notifications when a new reservation is made.

Guest Side

This side of the application is for customers who want to browse the restaurant menu and make reservations from their mobile device.

Functional Requirements – Guests

- As a guest, I wish to browse the menu items with details such as price, and images.
- As a guest, I wish to make a table reservation for a specific date and time.
- As a guest, I wish to edit or cancel my reservations.
- As a guest, I wish to receive notifications when my reservation status changes.

Additional Requirements

- The app must allow user authentication and role-based access control (staff and guest features must be accessible only to their respective roles).
- Notifications should be customisable so users can choose which updates they want to receive.
- The application interface must adapt to different screen sizes and orientations.

Assessment 1: Design Exercises

Task:

The design exercises consist of a series of structured exercises designed to assess your knowledge and understanding of user interface design, mobile applications, and usability evaluation. You will work with the coursework scenario provided on the top and produce designs for both types of users mentioned in the coursework scenario section.

Your work should be compiled into a single PDF document containing the outputs of all exercises in the same order they appear below.

The GitHub Classroom link for setting up your Android repository is provided below. Use this link to create your repository and maintain your application code throughout the module — from the initial UI design stage to the final version containing all required functionality.

GitHub Classroom: https://classroom.github.com/a/px0_XBKk

Exercise 1: 5 marks

- Analyse the scenario to identify the context of use and the intended users of the system.

Exercise 2: 20 marks

- Design the user interfaces for the mobile application using a low-fidelity prototype (e.g., paper sketches or basic digital wireframes).
- Include a storyboard to illustrate user interactions and task flows.
- Ensure the design follows the HCI principles covered during the semester.

Exercise 3: 30 marks

- Conduct a formative usability evaluation of the initial low-fidelity prototype of the application.
- Involve at least two representative users in the study.
- Use one of the testing methods discussed in the module, applying your knowledge to plan, conduct, and document the evaluation.

Exercise 4: 15 marks

- Re-design the user interfaces based on feedback from Exercise 3.
- Clearly highlight and explain the changes made.
- Produce a final storyboard with a clear narrative describing each interface from the user's perspective.

Set Exercise 5: 30 marks

- Implement the final UI design in Android Studio to create a high-fidelity design for one type of user.
- Use correct UI components to support future functionality.
- Include all pages/layouts for the final solution, but no back-end or functional Java code is required at this stage.

Submission Guidelines & Deliverables:

You are required to submit **three components** for this coursework:

1. Design Report (PDF format, 1500 words)

A single PDF document containing the outputs for all exercises (in order), clearly labelled and must include:

- Initial design of the UIs and the storyboard.
- Full details of the usability evaluation process, including how the study was conducted, the testing plan, participant demographics, consent form used outcomes of the study, evidence of the study setting (e.g., photos or screenshots).
- Final design based on user feedback, with differences from the initial design clearly highlighted and its storyboard.
- Final high-fidelity UI designed Android Studio with appropriate discussion.
- The link to the GitHub repository containing your source code.
- The 1500-word limit does not include references or appendices.

2. Video Presentation (MP4 format, 4–5 minutes)

- Provide a recorded walkthrough of your design process and key outcomes.
- The video should include:
 - An overview of the scenario and identified users' needs.
 - Highlights of your low-fidelity and final designs.
 - A brief summary of usability testing and how feedback informed changes.
- Save your video as an MP4 file and upload it to the DLE (or provide a shareable link).

3. GitHub Repository

- Use the GitHub Classroom link provided to create your repository.
- The repository should contain your Android Studio project for the implementation stage (Exercises 5). The repository should be maintain from the start of UI design through to the final submission of CW2.
- No changes to repository should be made after the submission deadline.

Acceptable level of generative AI tool use in this assessment element

The acceptable level of GenAI use for this element is detailed with the allowed uses listed below. This is split into three categories (**Solo Work** – work must be your own with no AI support, **Assisted Work** – some uses of AI tools allowed, **Partnered Work** – AI tools integral part of the work). If you have any questions, please contact the module leader.

Solo Work	You must not use generative AI tools.	☐
Assisted Work	You are permitted to use generative AI tools in an assistive role.	☒
Partnered Work	Generative AI tool use is required as an integral part of the assessment, but transparency is required.	☐

Table 2: Acceptable Level of generative AI use

Inline with the indicated acceptable level of AI use above, the following uses are acceptable:

- A1 – Idea Generation and Problem Exploration
- A2 - Planning & Structuring Projects
- A4 – Research Assistance
- A5 - Language Refinement
- A8 - Technical Guidance & Debugging Support

Any use of AI in your work must be declared within your documentation. You **must also include a signed Generative AI Declaration as an appendix to your submission**. The declaration form can be found on DLE (or Here on the Programme Page). This form will not be included in any word count associated with this assignment.

Threshold Criteria (these are indicative only):

Criteria	Fail (<40%)	3rd (40%+)	2:2 (50%+)	2:1 (60%+)	1st (70%+)
Response Content (40%)	The response outline fails to describe the core direction and/or lacks clarity of the arguments and flow of the response is illogical.	The response outline does provide the core direction of the report and has somehow described the main arguments. The given flow of the response is logical too.	The response outline provides a good direction and has identified good arguments. The given flow of the response is also logical.	A very good and concise response outline describing the main direction of the exercise with very good arguments. The flow of the response is very logical.	An excellent and concise response outline describing the main direction of the exercise with excellent arguments. The flow of the report is very logical.
Response Structure (30%)	No or illogical response structure has been given.	There is a response structure with some level 1/2 headings, but the text does give little or no details.	Report structure is good with almost all level 1/2 headings, and the text in these sections clearly shows how these sections will be populated.	Report structure is very good with almost all level 1/2 headings, and the text in these sections clearly shows how these sections will be populated.	The Report structure is excellent with a comprehensive list of level 1/2 headings, and the text in these sections robustly shows how these sections will be populated.
Presentation (30%)	Demo video structure, and style is poor. Submission elements are missing in demo.	The demo video style and structure is fine but occasionally incoherent. Presentation could have been improved.	The demo video style and structure are good and easily followed. Presentation is good.	The demo video style and structure is very good and easily followed. Presentation is very good.	The demo video style and structure is excellent and easily followed. Presentation is of excellent quality.
Feedback/Overall (100%)					

Table 3: Marking scheme for Assessment 1. This criterion is for each individual exercise of Assessment 1.

Assessment 2: Implementation

Task:

You are required to produce an Android application for the **Staff and Guest** sides, which will interact with a RESTful API and a local database. The application will allow staff to manage menu items and reservations, and guests to browse the menu and make reservations. The user stories reflecting requirements of both user types from the scenario are available in **coursework scenario** section at the front of this document.

The details of the API are available in the API documentation on the DLE page under Assessments tab. This file provides the root path and all available endpoints for the API.

Your application must:

- Be implemented in Java using Android Studio.
- Handle all connections to the API in a worker thread to avoid blocking the UI.
- Support notifications based on user-defined preferences.
- Apply the design patterns and SOLID principles covered in the course.
- Build upon the UI designs produced in your previous assessment as a foundation.

You are also required to:

- Carry out a summative usability evaluation with at least two test users.
- Ensure that the evaluation tests features relevant to both user roles.

The Graphical User Interface (GUI) should implement the functional requirements outlined in the scenario for both user types and must be responsive for different mobile device screen sizes. You **MUST** use Git version control with GitHub throughout your development process. You will be assessed on appropriate and consistent use of version control. Use the GitHub Classroom set up above for this module

Submission Guidelines & Deliverables:

You must submit the following for this coursework:

1. Implementation Report (PDF format, 3000 words)

- Include the design of the UIs (for both user types).
- The functionality should be clearly discussed separately according to task.
- A clear justification should be provided for different design choices.
- Provide full details of the final usability evaluation (how it was conducted, the testing plan, participant demographics, consent form used, outcomes, and evidence of the study setting).
- Provide the link to your GitHub repository containing the complete source code.
- Report should be organised in following main sections: Introduction, UI Design, Data management, Application control, Design practices and choices, Testing and LSEP and references.

2. Video Presentation (MP4 format, 4–6 minutes)

- Demonstrate your application in use, highlighting how it meets all requirements of the scenario for both user roles.
- Save your video as an MP4 file and upload it to the DLE (or provide a shareable link).
- The demo video is a compulsory component of submission.

3. GitHub Repository

- Hosted in the GitHub Classroom repository created using the module link.
- Must show incremental commits from the start of development to the final build.
- Include the complete Android Studio project and a README file listing all third-party resources used.
- No changes to repository should be made after the submission deadline.

Acceptable level of generative AI tool use in this assessment element

The acceptable level of GenAI use for this element is detailed with the allowed uses listed below. This is split into three categories (**Solo Work** – work must be your own with no AI support, **Assisted Work** – some uses of AI tools allowed, **Partnered Work** – AI tools integral part of the work). If you have any questions, please contact the module leader.

Solo Work	You must not use generative AI tools.	☐
Assisted Work	You are permitted to use generative AI tools in an assistive role.	☒
Partnered Work	Generative AI tool use is required as an integral part of the assessment, but transparency is required.	☐

Table 4: Acceptable Level of generative AI use

Inline with the indicated acceptable level of AI use above, the following uses are acceptable:

- A1 – Idea Generation and Problem Exploration
- A2 - Planning & Structuring Projects
- A4 – Research Assistance
- A5 - Language Refinement
- A8 - Technical Guidance & Debugging Support

Any use of AI in your work must be declared within your documentation. You **must also include a signed Generative AI Declaration as an appendix to your submission**. The declaration form can be found on DLE (or Here on the Programme Page). This form will not be included in any word count associated with this assignment.

Assessment Criteria:

The following marking scheme will be used for this part of the assessment:

- Report Outline and Introduction (5%)

- UI design (15%)
- Data management (20%)
 - Local and central database
- Application control (20%)
 - User specific app behaviour, authentication and notifications
- Design practices and choices (10%)
 - Reasoning behind different choices, use of design patterns and SOLID principles
- Testing and LSEP (15%)
 - Usability testing of final solution and Legal, Social, Ethical, and Professional (LSEP)
- Presentation (15%)
 - Demo video is a compulsory part of the submission similar to GitHub. Absence of video will impact above component due to absence of evidence.

Criteria	Fail (<40%)	3rd (40%–49%)	2:2 (50%–59%)	2:1 (60%–69%)	1st (70%+)
Report Outline and Introduction (5%)	No meaningful introduction or outline. Very poor report structure/readability.	Very limited introduction; unclear objectives; poor formatting.	Basic introduction with some outline of aims; formatting adequate but weak.	Clear introduction with good structure and objectives; generally well presented.	Comprehensive, well-structured introduction with excellent readability and formatting.
UI Design (15%) (<i>UI + Interaction Design Principles</i>)	Weak or incomplete UI; no evidence of interaction design principles. <i>Quality of demo video will impact this section.</i>	Basic UI with major usability issues; limited application of interaction design principles.	Functional UI with some usability but weak consistency/interaction design.	Clear, usable UI applying several interaction design principles; minor issues with consistency or navigation.	Intuitive, creative, and user-friendly UI; excellent application of UI and interaction design principles (consistency, feedback, accessibility, navigation). Strong alignment with previous assessment designs. Report is easy to follow and understand.
Data Management (20%) (<i>Local and central database</i>)	Incomplete or missing database implementation. <i>Quality of demo video will impact this section.</i>	Limited database functionality; poor integration.	Basic functionality; some issues with consistency/security.	Functional integration with minor inefficiencies.	Robust local and central database integration; efficient CRUD operations; secure handling of data. This section is clearly explained in report and is easy to follow.
Application Control (20%) (<i>User behaviour, authentication, notifications</i>)	Missing or very weak control features. <i>Quality of demo video will impact this section.</i>	Basic features present but poorly integrated.	Some user-specific behaviour; partial authentication/notifications.	Functional authentication and notifications; generally consistent user behaviour.	Comprehensive implementation of authentication, user-specific behaviour, and notifications; seamless integration. The content of this section is clearly explained, and section is easy to follow.
Design Practices & Choices (10%) (<i>Patterns, SOLID principles</i>)	No justification; no use of patterns/principles. Very poor report readability.	Minimal reasoning; very weak application of principles.	Some reasoning provided; limited application of principles.	Clear reasoning with adequate application of design patterns and SOLID principles.	Strong justification of choices; excellent use of design patterns and SOLID principles. Well-structured explanations in report required.
Testing and LSEP (15%) (<i>Usability + Legal, Social, Ethical, Professional</i>)	No testing or LSEP considerations; report unreadable.	Minimal testing; weak LSEP discussion.	Some usability testing; limited LSEP reflection.	Usability testing with at least two users; some analysis of LSEP issues.	Thorough usability testing with detailed analysis; strong reflection on LSEP issues. This section is clearly explained in report and is easy to follow.
Presentation (15%) (<i>Demo video + GitHub</i>)	No demo video and/or poor/absent GitHub submission.	Demo video or GitHub present but incomplete or very weak quality.	Demo video/GitHub provided but lacks clarity or completeness.	Clear demo video and well-structured GitHub repository.	Excellent, professional-quality demo video and GitHub submission; strong evidence of functionality.
Overall (100%)					

Table 5: Marking scheme for Assessment 2

Acceptable levels of AI use:

The table below provides the acceptable use categories for GenAI. Each assessment element may allow different uses. Please check the brief for each element carefully to see what uses are allowed.

Solo Work	S1 - Generative AI tools have not been used for this assessment.
Assisted Work	A1 – Idea Generation and Problem Exploration Used to generate project ideas, explore different approaches to solving a problem, or suggest features for software or systems. Students must critically assess AI-generated suggestions and ensure their own intellectual contributions are central.
	A2 - Planning & Structuring Projects AI may help outline the structure of reports, documentation and projects. The final structure and implementation must be the student's own work.
	A3 – Code Architecture AI tools may be used to help outline code architecture (e.g. suggesting class hierarchies or module breakdowns). The final code structure must be the student's own work.
	A4 – Research Assistance Used to locate and summarise relevant articles, academic papers, technical documentation, or online resources (e.g. Stack Overflow, GitHub discussions). The interpretation and integration of research into the assignment remain the student's responsibility.
	A5 - Language Refinement Used to check grammar, refine language, improve sentence structure in documentation not code. AI should be used only to provide suggestions for improvement. Students must ensure that the documentation accurately reflects the code and is technically correct.
	A6 – Code Review AI tools can be used to check comments within the code and to suggest improvements to code readability, structure or syntax. AI should be used only to provide suggestions for improvement. Students must ensure that the code accurately reflects their knowledge and is technically correct.
	A7 - Code Generation for Learning Purposes Used to generate example code snippets to understand syntax, explore alternative implementations, or learn new programming paradigms. Students must not submit AI-generated code as their own and must be able to explain how it works.
	A8 - Technical Guidance & Debugging Support AI tools can be used to explain algorithms, programming concepts, or debugging strategies. Students may also help interpret error messages or suggest possible fixes. However, students must write, test, and debug their own code independently and understand all solutions submitted.
	A9 - Testing and Validation Support AI may assist in generating test cases, validating outputs, or suggesting edge cases for software testing. Students are responsible for designing comprehensive test plans and interpreting test results.
	A10 - Data Analysis and Visualization Guidance AI tools can help suggest ways to analyse datasets or visualize results (e.g. recommending chart types or statistical methods). Students must perform the analysis themselves and understand the implications of the results.
	A11 - Other uses not listed above Please specify:
Partnered Work	P1 - Generative AI tool usage has been used integrally for this assessment Students can adopt approaches that are compliant with instructions in the assessment brief. Please Specify:

General Guidance

Extenuating Circumstances

There may be a time during this module where you experience a serious situation which has a significant impact on your ability to complete the assessments. The definition of these can be found in the University Policy on Extenuating Circumstances here:

<https://www.plymouth.ac.uk/student-life/your-studies/essential-information/exams/exam-rules-and-regulations/extenuating-circumstances>

Plagiarism

All of your work must be of your own words. You must use references for your sources, however you acquire them. Where you wish to use quotations, these must be a very minor part of your overall work.

To copy another person's work is viewed as plagiarism and is not allowed. Any issues of plagiarism and any form of academic dishonesty are treated very seriously. All your work must be your own and other sources must be identified as being theirs, not yours. The copying of another persons' work could result in a penalty being invoked.

Further information on plagiarism policy can be found here:

Plagiarism: <https://www.plymouth.ac.uk/student-life/your-studies/essential-information/regulations/plagiarism>

Examination Offences: <https://www.plymouth.ac.uk/student-life/your-studies/essential-information/exams/exam-rules-and-regulations/examination-offences>

Turnitin (<http://www.turnitinuk.com/>) is an Internet-based 'originality checking tool' which allows documents to be compared with content on the Internet, in journals and in an archive of previously submitted works. It can help to detect unintentional or deliberate plagiarism.

It is a formative tool that makes it easy for students to review their citations and referencing as an aid to learning good academic practice. Turnitin produces an 'originality report' to help guide you. To learn more about Turnitin go to:

<https://help.turnitin.com/new-links.htm?Highlight=guide>

The university also had Draft Coach available on the online version of Microsoft Word. Draft Coach is a Turnitin plugin and provides instant feedback on how to address citation issues, grammar mistakes and matches with Turnitin's database.

Referencing

The University of Plymouth Library has produced an online support referencing guide which is available here: <http://plymouth.libguides.com/referencing>.

Another recommended referencing resource is [Cite Them Right Online](#); this is an online resource which provides you with specific guidance about how to reference lots of different types of materials.